

Javascript och WebAPI

async och await?

JavaScript är **enkeltrådat**, vilket innebär att det bara kan utföra en instruktion åt gången. Om vi skickar ett nätverksanrop över HTTP synkront, blockeras hela tråden. Webbläsaren fryser och gränssnittet slutar svara under de millisekunder (eller sekunder) det tar för servern och databasen att svara.

För att lösa detta använder vi asynkron programmering med **async** och **await**:

1. **async**: Placeras framför en funktionsdeklaration. Det transformerar funktionen så att den automatiskt returnerar ett **Promise** (ett objekt som representerar en asynkron operation som kommer att slutföras i framtiden).
2. **await**: Placeras inuti en asynkron funktion, precis framför det anrop som tar tid (som `fetch()` eller `.json()`). Det pausar exekveringen *inuti just den funktionen* tills operationen är klar, men frigör samtidigt JavaScripts huvudtråd så att resten av webbsidan (rendering, skrollning, klick) fortsätter att rulla helt fritt.

De 4 CRUD-varianterna: Tekniskt flöde och kodkonstruktion

Här bryter vi ner exakt vad som händer under huven för varje HTTP-metod i gränssnittet.

1. READ (Hämta data – GET)

När du hämtar data skickar du en tom HTTP-request till servern. Serverns uppgift är att läsa från SQLite via EF Core och serialisera databasraderna till en JSON-array.

- **HTTP-metod:** GET
- **URL:** <https://localhost:7187/api/bilar>
- **Förväntad statuskod:** 200 OK
- **Tekniskt flöde i JavaScript:**
 - `fetch(API_URL)` skickas och vi väntar (`await`) på HTTP-headern från servern.
 - `response.json()` anropas för att strömma och parsas textmassan (JSON) från nätverket till en hanterbar array av JavaScript-objekt. Denna operation är också asynkron och kräver därför `await`.

```
const fetchCars = async () => {
  const response = await fetch(API_URL); // Väntar på
  nätverksrespons
  const cars = await response.json(); // Väntar på att texten
  konverteras till JS-objekt
  // Nu kan vi loopa igenom 'cars' och uppdatera DOM:en
};
```

2. CREATE (Skapa ny data – POST)

När du skapar en ny bil skickar du med data i anropets **Request Body**. Backend tar emot denna JSON-sträng, mappar den mot C#-klassen Car, tilldelar ett nytt unikt ID via SQLite (Autoincrement) och sparar raden.

- **HTTP-metod:** POST
- **URL:** <https://localhost:7187/api/bilar>
- **Förväntad statuskod:** 201 Created
- **Tekniskt flöde i JavaScript:**
 - Formulärets standard-submit avbryts med `event.preventDefault()`.
 - Formulärdata samlas in (strängar konverteras till heltal med `parseInt()`).
 - `fetch` konfigureras med ett inställningsobjekt där `method` sätts till `'POST'`, `Content-Type` sätts till `'application/json'` och JavaScript-objektet strängifieras med `JSON.stringify()`.

```
const response = await fetch(API_URL, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(newCar) // Gör om JS-objekt till JSON-
  text
});
if (response.ok) {
  // Rensa formulär och anropa fetchCars() för att uppdatera
  listan
}
```

3. UPDATE (Uppdatera befintlig data – PUT)

Att uppdatera innebär att du skriver över en redan existerande rad i databasen. För att servern ska veta vilken rad som ska ändras måste du skicka med bilens unika ID på två ställen: i URL:en (så att .NET hittar rätt endpoint-rutt) och oftast i din JSON-body (för att matcha databas-entiteten under sparningen).

- **HTTP-metod:** PUT
- **URL:** <https://localhost:7187/api/bilar/{id}> (t.ex. /api/bilar/3)
- **Förväntad statuskod:** 204 NoContent (eller 200 OK)
- **Tekniskt flöde i JavaScript:**
 - Användaren klickar på "Redigera" och bilens befintliga data flyttas till formulärfälten. Bilens ID sparas i ett dolt fält eller en variabel.
 - När formuläret sparas skickas en PUT-request till den specifika URL:en.
 - Innehållet i body måste innehålla de uppdaterade värdena samt bilens ID.

```
const id = 3; // ID på bilen som ska uppdateras
const response = await fetch(`${API_URL}/${id}`, {
  method: 'PUT',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(updatedCarData)
});
```

4. DELETE (Radera data – DELETE)

Radering är ett mycket lättviktigt anrop. Du skickar inte med någon JSON-body överhuvudtaget, utan identifierar resursen helt och hållet via URL-strukturen. Servern kör `db.Cars.Remove()` baserat på ID:t och verkställer raderingen i databasfilen.

- **HTTP-metod:** DELETE
- **URL:** <https://localhost:7187/api/bilar/{id}> (t.ex. /api/bilar/3)
- **Förväntad statuskod:** 200 OK (eller 204 NoContent)
- **Tekniskt flöde i JavaScript:**
 - När raderingsknappen trycks in skickas bilens id med till den asynkrona funktionen.
 - `fetch` anropas med en sammansatt URL (API-basadress + ID).
 - `method` sätts till 'DELETE'. Inga headers för datatyp eller bodies behövs eftersom ingen data skickas med.

```
const deleteCar = async (id) => {
  const response = await fetch(`${API_URL}/${id}`, {
    method: 'DELETE' // Informerar servern om att radera
    resursen på denna URL
  });
  if (response.ok) {
    // Anropet lyckades, kör fetchCars() för att rita ut den
    uppdaterade listan
  }
};
```

Sammanfattande tabell över HTTP-kommunikationen

CRUD Operation	HTTP Metod	URL-struktur	Skickar Request Body?	Typisk Statuskod från .NET
Create	POST	/api/bilar	Ja (Hela objektet som JSON)	201 Created
Read	GET	/api/bilar eller /api/bilar/{id}	Nej	200 OK
Update	PUT	/api/bilar/{id}	Ja (Uppdaterat objekt som JSON)	204 No Content
Delete	DELETE	/api/bilar/{id}	Nej	200 OK 204 No Content